Certifying every runtime step of the agent: a zkPoP skill, self-hosting, and a generated proof

Machine-checked in Lean 4 with Mathlib
`RequestProject/StepZK/`

Abstract

The zero-knowledge machinery of this repository has so far certified *static* objects: the Lean modules of the project, and the pi-agent port. This paper certifies the agent *while it runs*. A zkPoP skill is attached to each runtime step: it emits one certificate per step, and the certificate is accepted exactly when the step really is the operation the runtime semantics prescribes, leaves the session history well formed, grows it by at most 2 entries, and does not lose ground on the codebase the agent is writing. The certificates of a run compose, and out of them *self-hosting* is proved: any certified run of review-and-improve tool calls, long enough to cover the defects the draft still has, ends with the intended codebase. On the concrete run of this paper (4 certified steps, 3 defects before, 0 after, history $1 \to 9$ entries) the skill then *generates a proof*: a Lean file with one theorem per certified step, re-checked from scratch by the Lean kernel when the library is built. The run is arithmetised into 5 rank-1 constraints and committed to; published run digest 5279 (mod 10 007). No step of the argument uses an axiom beyond Lean's own `propext`, `Classical.choice` and `Quot.sound`.

1 The object: one runtime step of the agent

The agent's runtime state is the pair of what it carries between steps: the session history of the agent loop (`RequestProject/AgentLoop.lean`) and the draft codebase it is rewriting into Lean, as the windows of code of `RequestProject/SelfHostSim.lean`.

```
structure RtState where
  /-- the session history (the state of the agent loop) -/
  hist : List SessionTreeEntry
  /-- the draft codebase, as a list of windows of code -/
  draft : List Window
  deriving DecidableEq
```

A runtime operation is either a plain agent-loop action on the history — a user turn, an assistant message, a tool result, or a context compaction — or a review-and-improve pass, the tool call in which the agent improves its own draft. A review pass *is* a tool call, so it moves the history too: the assistant message that requests it, then the tool result it returns.

```
inductive Op
  /-- an agent-loop action: append a turn, or compact the context -/
  | hist (a : Action)
  /-- a review-and-improve pass on the draft (a tool call) -/
  | review
  deriving DecidableEq, Repr


def Op.actions : Op → List Action
  | .hist a => [a]
  | .review => [.assistant, .toolResult]
```

```
def stepState (estTok : AgentMessage → ℕ) (target : List Window) :
    Op → RtState → RtState
  | .hist a, s => ⟨applyAction estTok a s.hist, s.draft⟩
  | .review, s =>
      ⟨applyAction estTok .toolResult (applyAction estTok .assistant s.hist),
        reviewStep target s.draft⟩
```

2 The skill: what it certifies about a step

The declaration a step is checked against, and the certificate the skill emits for one step:

```
structure StepClaim where
  /-- if set, the session history must be well formed before and after the step -/
  requireWF : Bool
  /-- the step may append at most this many entries to the history -/
  growth : ℕ
  /-- if set, the step must not increase the number of remaining defects -/
  requireProgress : Bool
  deriving DecidableEq, Repr


structure StepCert where
  /-- the position of the step in the run -/
  idx : ℕ
  /-- the operation performed -/
  op : Op
  /-- the runtime state before the step -/
  pre : RtState
  /-- the runtime state after the step -/
  post : RtState
  deriving DecidableEq
```

Its semantic reading and the skill's own Boolean check are proved equivalent, which is what makes the claim decidable — and hence arithmetisable:

```
def StepCert.Meets (estTok : AgentMessage → ℕ) (target : List Window)
    (s : StepCert) (c : StepClaim) : Prop :=
  s.post = stepState estTok target s.op s.pre ∧
    (c.requireWF = true → WF s.pre.hist ∧ WF s.post.hist) ∧
    s.post.hist.length ≤ s.pre.hist.length + c.growth ∧
    (c.requireProgress = true →
      defects target s.post.draft ≤ defects target s.pre.draft)


theorem StepCert.eval_iff (estTok : AgentMessage → ℕ) (target : List Window)
    (s : StepCert) (c : StepClaim) :
    s.eval estTok target c = true ↔ s.Meets estTok target c := by
  cases hw : c.requireWF <;> cases hp : c.requireProgress <;>
    simp [StepCert.eval, StepCert.Meets, hw, hp, and_assoc]
```

The skill is complete for genuine runtime steps: the certificate read off an actual step of the agent always passes.

```
theorem certOf_meets (estTok : AgentMessage → ℕ) (target : List Window)
    (i : ℕ) (o : Op) (s : RtState) (c : StepClaim)
    (hwf : c.requireWF = true → WF s.hist) (hg : o.cost ≤ c.growth) :
    (certOf estTok target i o s).Meets estTok target c := by
  refine ⟨rfl, ?_, ?_, ?_⟩
    ...
```

3 A run, and what its certificates establish

A run is a list of certificates that chain: the first starts at the initial state and each starts where its predecessor ended. The content of the composition is that the certificates *pin the trajectory* — a certified run really is the agent's own execution:

```
theorem sound_chain_replays (estTok : AgentMessage → ℕ) (target : List Window)
    (c : StepClaim) :
    ∀ (xs : List StepCert) (s : RtState), chainFrom s xs →
      (∀ x ∈ xs, x.Meets estTok target c) →
      finalOf s xs = runOps estTok target (xs.map StepCert.op) s
  | [], s, _, _ => rfl
    ...
```

Everything else about the run follows from that: the history stays well formed, it grows by at most two entries per step, and the draft never loses ground.

```
theorem chain_WF (estTok : AgentMessage → ℕ) (target : List Window) (c : StepClaim)
    (t : Trace) (s : RtState) (hchain : t.From s)
    (hsound : t.Sound estTok target c) (hwf : WF s.hist) :
    WF (t.final s).hist := by
  rw [Trace.final, sound_chain_replays estTok target c t.steps s hchain hsound]
    ...


theorem chain_length_le (estTok : AgentMessage → ℕ) (target : List Window) (c : StepClaim)
    (t : Trace) (s : RtState) (hchain : t.From s) (hsound : t.Sound estTok target c) :
    (t.final s).hist.length ≤ s.hist.length + 2 * t.size := by
  rw [Trace.final, sound_chain_replays estTok target c t.steps s hchain hsound]
    ...


theorem chain_defects_le (estTok : AgentMessage → ℕ) (target : List Window) (c : StepClaim)
    (t : Trace) (s : RtState) (hchain : t.From s) (hsound : t.Sound estTok target c) :
    defects target (t.final s).draft ≤ defects target s.draft := by
  rw [Trace.final, sound_chain_replays estTok target c t.steps s hchain hsound]
    ...
```

4 Self-hosting, proved with the skill

`RequestProject/SelfHostSim.lean` models the agent self-hosting: it writes its codebase window by window, then reviews and improves the draft, one pass per tool call. Here that run is executed *through the skill*, and self-hosting is obtained from the certificates alone: nothing is assumed about the run beyond what each per-step certificate says.

```
theorem selfHost_of_certified (estTok : AgentMessage → ℕ) (target : List Window)
    (c : StepClaim) (t : Trace) (s : RtState)
    (hchain : t.From s) (hsound : t.Sound estTok target c)
    (hops : t.ops = reviewOps t.size)
    (hlen : target.length = s.draft.length)
    (hn : defects target s.draft ≤ t.size) :
    (t.final s).draft = target := by
  have hreplay : t.final s = runOps estTok target t.ops s := by
    rw [Trace.final, Trace.ops]
    ...
```

The skill run alongside the agent emits exactly such a trace, so it self-hosts the agent, with the well-formedness and growth guarantees attached:

```
theorem selfHost_certified (estTok : AgentMessage → ℕ) (target : List Window)
    (c : StepClaim) (n : ℕ) (s : RtState)
    (hg : 2 ≤ c.growth) (hwf : WF s.hist)
    (hlen : target.length = s.draft.length) (hn : defects target s.draft ≤ n) :
    (certify estTok target (reviewOps n) s).Sound estTok target c ∧
      (certify estTok target (reviewOps n) s).From s ∧
      (certify estTok target (reviewOps n) s).size = n ∧
      ((certify estTok target (reviewOps n) s).final s).draft = target ∧
      WF ((certify estTok target (reviewOps n) s).final s).hist ∧
      ((certify estTok target (reviewOps n) s).final s).hist.length ≤ s.hist.length + 2 * n := by
  have hsize : (certify estTok target (reviewOps n) s).size = n := by simp
  have hsound : (certify estTok target (reviewOps n) s).Sound estTok target c :=
    ...
```

And once self-hosted, the codebase reproduces itself:

```
theorem selfHost_stable_certified (estTok : AgentMessage → ℕ) (target : List Window)
    (c : StepClaim) (t : Trace) (s : RtState)
    (hchain : t.From s) (hsound : t.Sound estTok target c)
    (hops : t.ops = reviewOps t.size) (hs : s.draft = target) :
    (t.final s).draft = target := by
  have hreplay : t.final s = runOps estTok target t.ops s := by
    rw [Trace.final, Trace.ops]
    ...
```

On the concrete run: 5 windows of intended code, 3 of them still wrong in the draft, 4 review-and-improve tool calls, 0 defects left, and a session history that grew from 1 to 9 entries. Every step certified:

step	operation	hist. before	after	defects before	after	certified	digest
0	`review pass`	1	3	3	2	yes	1548
1	`review pass`	3	5	2	1	yes	10004
2	`review pass`	5	7	1	0	yes	6880
3	`review pass`	7	9	0	0	yes	4307

The other runtime operations are certified too. The second run below (6 steps, digest 329) interleaves a user turn, an assistant message and a tool result with review passes. Context compaction — the fourth agent-loop action — is certified for *every* history at once rather than on an example, because the verified cut-point selector it calls is defined by well-founded recursion and so is not unfolded by the kernel:

```
theorem compaction_step_certified (estTok : AgentMessage → ℕ) (target : List Window)
    (i k : ℕ) (s : RtState) (hwf : WF s.hist) :
    (certOf estTok target i (.hist (.compact k)) s).Meets estTok target demoClaim :=
  certOf_meets estTok target i _ s demoClaim (fun _ => hwf)
    (le_of_eq_of_le (Op.cost_hist _) (by decide))
```

step	operation	hist. before	after	defects before	after	certified	digest
0	`user turn`	1	2	3	3	yes	6857
1	`assistant message`	2	3	3	3	yes	1246
2	`tool result`	3	4	3	3	yes	5642
3	`review pass`	4	6	3	2	yes	2456
4	`review pass`	6	8	2	1	yes	905
5	`review pass`	8	10	1	0	yes	7788

The skill is not a rubber stamp: a certificate that reports a state the runtime semantics does not produce is rejected, by kernel computation.

```
theorem demo_forgery_rejected : ¬ demoForgery.Meets demoEstTok demoTarget demoClaim := by decide
```

5 The skill hosts itself

Certifying a step is itself something the agent *does*: a tool call, hence an assistant message and a tool result. So the skill's own execution is a list of runtime operations, and those operations are certified by the same skill.

```
def auditOps (xs : List StepCert) : List Op :=
  xs.flatMap (fun _ => [Op.hist .assistant, Op.hist .toolResult])


theorem auditTrace_sound (estTok : AgentMessage → ℕ) (target : List Window)
    (c : StepClaim) (hg : 2 ≤ c.growth) (t : Trace) (s : RtState) (hwf : WF s.hist) :
    (auditTrace estTok target t s).Sound estTok target c ∧
      (auditTrace estTok target t s).From s ∧
      (auditTrace estTok target t s).size = 2 * t.size :=
  ⟨certify_sound estTok target _ s c hg (fun _ => hwf),
    ...
```

Iterating — certifying the certification of the certification, to any depth — never leaves the certified fragment:

```
theorem audit_iter_sound (estTok : AgentMessage → ℕ) (target : List Window)
    (c : StepClaim) (hg : 2 ≤ c.growth) (k : ℕ) (p : Trace × RtState) (hwf : WF p.2.hist) :
    ((auditStep estTok target)^[k + 1] p).1.Sound estTok target c := by
  rw [Function.iterate_succ_apply']
    ...
```

Concretely, certifying the 4-step run costs 8 runtime steps, all certified; certifying *those* costs 16, all certified.

6 The check verifies its own execution

The run's check is arithmetised as a gate circuit, satisfiable exactly when every step of the run is certified:

```
def traceGates (estTok : AgentMessage → ℕ) (target : List Window)
    (t : Trace) (c : StepClaim) : GateCircuit F :=
  (List.range t.size).flatMap
    (fun i => [Gate.constG (i + 1) (verdict estTok target t c i), Gate.constG (i + 1) (1 : F)])


theorem traceGates_satisfiable_iff (estTok : AgentMessage → ℕ) (target : List Window)
    (t : Trace) (c : StepClaim) :
    (∃ w : ℕ → F, GateCircuit.Sat w (traceGates estTok target t c)) ↔
      t.Sound estTok target c := by
  constructor
  · rintro ⟨w, hw⟩
    ...
```

Handing that circuit to the universal verifier of `RequestProject/ZkPoP/SelfHost.lean` gives the recursive step: one uniform constraint checks the run, and the verifier's own execution is checked by the same constraint.

```
theorem stepZK_two_level (estTok : AgentMessage → ℕ) (target : List Window)
    (t : Trace) (c : StepClaim) (h : t.Sound estTok target c) :
    (∀ r ∈ univCircuit (traceGates estTok target t c) (oneWire : ℕ → F), r.Sat) ∧
      (∀ r ∈ univCircuit (traceGates estTok target t c) (oneWire : ℕ → F),
        ∃ v : ℕ → F, (∀ i < 8, v i = r.assign i) ∧
          ∀ s ∈ univCircuit rowCircuit v, s.Sat) := by
  have hsat : GateCircuit.Sat (oneWire : ℕ → F) (traceGates estTok target t c) :=
    traceGates_sat_oneWire estTok target t c h
    ...
```

7 Arithmetisation of the whole run

For the zero-knowledge argument the run becomes a rank-1 constraint system: one booleanity constraint per certified step (4 of them) plus one counting constraint, 5 in total, over $\mathbb{Z}/10\,007\mathbb{Z}$.

```
def stepR1CS (estTok : AgentMessage → ℕ) (target : List Window)
    (t : Trace) (c : StepClaim) : R1CS F :=
  boolConstraints t.size ++
    [eqConstConstraint (cellsLC (t.acceptedIdx estTok target c)) (t.size : F)]
```

Over a field with more residues than the run has steps, the system is satisfiable exactly when every runtime step meets the declaration:

```
theorem stepR1CS_satisfiable_iff (estTok : AgentMessage → ℕ) (target : List Window)
    (t : Trace) (c : StepClaim) (hq : t.size < q) :
    (∃ w : ℕ → ZMod q, w 0 = 1 ∧ R1CS.Sat w (stepR1CS estTok target t c)) ↔
      t.Sound estTok target c := by
  constructor
  · rintro ⟨w, h0, hsat⟩
    ...
```

8 The zero-knowledge certificate of the run

The prover commits to the run's digest vector (5279, 9458, 40, 329) — the digest of the certified run, of the operations it certifies, of the state it ends in, and of the mixed run — blinded by fresh randomness, and applies the Fiat–Shamir transform. The committed vector is identified with the digests the kernel computes from the certificates themselves:

```
theorem stepVec_eq_digests :
    stepVec = ![((traceDigest demoTrace : ℕ) : K), ((opsDigest demoTrace : ℕ) : K),
      ((stateDigest demoFinal : ℕ) : K), ((traceDigest demoMixedTrace : ℕ) : K)] := by
  obtain ⟨h0, h1, h2, h3⟩ := demo_digests
  rw [h0, h1, h2, h3]
    ...
```

The published statement is (8488, 6312), the first message (17, 68), the challenge 7869, and the response (1395, 2947, 4544, 7099) with blinding (2985, 9338). The verification equation holds by kernel computation, and the published data are exactly the honest prover's output:

```
theorem stepCert_verifies :
    FSVerify (comMap phi psi) hash statement firstMessage response := by
  show comMap phi psi response = firstMessage + challenge • statement
  decide
```

```
theorem stepCert_data :
    statement = ![8488, 6312] ∧ firstMessage = ![17, 68] ∧ challenge = 7869 ∧
      response.1 = ![1395, 2947, 4544, 7099] ∧ response.2 = ![2985, 9338] := by
  refine ⟨by decide, by decide, by decide, by decide, by decide⟩
```

Two accepting certificates on one first message under different hash values yield an opening of the commitment — knowledge; and the joint law of the commitment and the whole interaction does not depend on which digest vector was committed to — zero knowledge. What is proved is that every step of the agent's run was certified; *what* the agent did stays hidden.

```
theorem stepCert_extraction (H H' : (Fin 2 → K) → (Fin 2 → K) → K)
    (z z' : (Fin 4 → K) × (Fin 2 → K))
    (h : FSVerify (comMap phi psi) H statement firstMessage z)
    (h' : FSVerify (comMap phi psi) H' statement firstMessage z')
    (hne : H statement firstMessage ≠ H' statement firstMessage) :
    comMap phi psi
        (extract (H statement firstMessage) (H' statement firstMessage) z z') = statement :=
  fs_extraction (comMap phi psi) H H' statement firstMessage z z' h h' hne


theorem stepCert_hides_run (v v' : Fin 4 → K) (e : K) :
    jointView phi psi v e = jointView phi psi v' e :=
  joint_view_indep_of_witness phi psi psi_surjective v v' e
```

9 The skill generates a proof

Having certified the run, the skill writes a Lean proof of what it found. `lake exe stepzk --proof` emits `RequestProject/StepZK/Generated.lean`: one theorem per certified runtime step, the digests it computed, and the outcome of the run — each closed by `decide`, that is, re-checked from scratch by the Lean kernel when the library is built. The generated file, verbatim (its header comment elided):

```
import RequestProject.StepZK.Final

/-!
# The generated proof of the certified run

This file was written by the skill itself (`RequestProject/StepZK/Emit.lean`,
`lake exe stepzk --proof`) after running over the agent's 4 runtime steps.
Each theorem below is the skill's own verdict, re-checked from scratch by the
Lean kernel.
-/

namespace StepZK

namespace Generated

open LemmaScript LemmaScript.AgentLoop LemmaScript.SelfHostSim

/-- Step 0: review pass (history 1 → 3, defects 3 → 2, digest 1548). -/
theorem step_0_certified : demoTrace.accepts demoEstTok demoTarget demoClaim 0 = true := by decide

/-- Step 1: review pass (history 3 → 5, defects 2 → 1, digest 10004). -/
theorem step_1_certified : demoTrace.accepts demoEstTok demoTarget demoClaim 1 = true := by decide

/-- Step 2: review pass (history 5 → 7, defects 1 → 0, digest 6880). -/
theorem step_2_certified : demoTrace.accepts demoEstTok demoTarget demoClaim 2 = true := by decide

/-- Step 3: review pass (history 7 → 9, defects 0 → 0, digest 4307). -/
theorem step_3_certified : demoTrace.accepts demoEstTok demoTarget demoClaim 3 = true := by decide
```

```
/-- The skill accepted every step of the run. -/
theorem run_certified : demoTrace.Sound demoEstTok demoTarget demoClaim := by decide

/-- The digest of the run, as the skill computed it. -/
theorem run_digest : traceDigest demoTrace = 5279 := by decide

/-- The digest of the operations the run certifies. -/
theorem run_ops_digest : opsDigest demoTrace = 9458 := by decide

/-- The certified run self-hosted: it ended with the intended codebase. -/
theorem run_self_hosted : demoFinal.draft = demoTarget := by decide

/-- No defect is left, and the final history is well formed. -/
theorem run_final_clean :
    defects demoTarget demoFinal.draft = 0 ∧ WF demoFinal.hist := by
  refine ⟨by decide, by decide⟩

/-- **The generated proof.**  Every runtime step of the agent was certified, the
run's digests are the published ones, and the run self-hosted. -/
theorem generated_proof :
    (∀ i < 4, demoTrace.accepts demoEstTok demoTarget demoClaim i = true) ∧
      demoTrace.Sound demoEstTok demoTarget demoClaim ∧
      traceDigest demoTrace = 5279 ∧ opsDigest demoTrace = 9458 ∧
      demoFinal.draft = demoTarget ∧ defects demoTarget demoFinal.draft = 0 :=
  ⟨by decide, run_certified, run_digest, run_ops_digest, run_self_hosted,
    run_final_clean.1⟩

end Generated

end StepZK
```

This is the sense in which the certification is a proof and not a report: the artifact the skill produces is checked by the same kernel that checks everything else in this repository, and it is checked *again* on every build.

10 Everything in one statement

The layers are collected into a single theorem: faithful arithmetisation, every runtime step certified, self-hosting achieved, the skill certifying its own steps to any depth, the check verifying its own execution, the identification of the committed vector with the measured digests, verification, honesty of the transcript, knowledge, and zero knowledge.

```
theorem stepZK_end_to_end :
    ((∃ w : ℕ → Certificate.K, w 0 = 1 ∧
        R1CS.Sat w (stepR1CS demoEstTok demoTarget demoTrace demoClaim)) ↔
      demoTrace.Sound demoEstTok demoTarget demoClaim) ∧
    demoTrace.Sound demoEstTok demoTarget demoClaim ∧
    demoFinal.draft = demoTarget ∧
    WF demoFinal.hist ∧
    (∀ (k : ℕ) (p : Trace × RtState), WF p.2.hist →
      ((auditStep demoEstTok demoTarget)^[k + 1] p).1.Sound demoEstTok demoTarget demoClaim) ∧
    ((∀ r ∈ ZkPoP.univCircuit
          (traceGates demoEstTok demoTarget demoTrace demoClaim) (oneWire : ℕ → Certificate.K),
        r.Sat) ∧
      (∀ r ∈ ZkPoP.univCircuit
          (traceGates demoEstTok demoTarget demoTrace demoClaim) (oneWire : ℕ → Certificate.K),
        ∃ v : ℕ → Certificate.K, (∀ i < 8, v i = r.assign i) ∧
          ∀ s ∈ ZkPoP.univCircuit ZkPoP.rowCircuit v, s.Sat)) ∧
    Certificate.stepVec = ![((traceDigest demoTrace : ℕ) : Certificate.K),
      ((opsDigest demoTrace : ℕ) : Certificate.K),
```

```
  ((stateDigest demoFinal : ℕ) : Certificate.K),
  ((traceDigest demoMixedTrace : ℕ) : Certificate.K)] ∧
FSVerify (comMap Certificate.phi Certificate.psi) Certificate.hash
  Certificate.statement Certificate.firstMessage Certificate.response ∧
(Certificate.firstMessage, Certificate.challenge, Certificate.response) =
  prover (comMap Certificate.phi Certificate.psi) Certificate.witness
    Certificate.randomness Certificate.challenge ∧
(∀ (H H' : (Fin 2 → Certificate.K) → (Fin 2 → Certificate.K) → Certificate.K)
    (z z' : (Fin 4 → Certificate.K) × (Fin 2 → Certificate.K)),
  FSVerify (comMap Certificate.phi Certificate.psi) H Certificate.statement
      Certificate.firstMessage z →
    FSVerify (comMap Certificate.phi Certificate.psi) H' Certificate.statement
        Certificate.firstMessage z' →
      H Certificate.statement Certificate.firstMessage ≠
          H' Certificate.statement Certificate.firstMessage →
        comMap Certificate.phi Certificate.psi
            (extract (H Certificate.statement Certificate.firstMessage)
              (H' Certificate.statement Certificate.firstMessage) z z') =
          Certificate.statement) ∧
(∀ (v v' : Fin 4 → Certificate.K) (e : Certificate.K),
  jointView Certificate.phi Certificate.psi v e =
    jointView Certificate.phi Certificate.psi v' e) :=
...
```

11 Do we have the proof? The axiom audit

A Lean proof is only as good as what it rests on. `RequestProject/StepZK/Audit.lean` prints the transitive axiom closure of every headline result; this is that output, verbatim. No `sorry` occurs anywhere in the layer, and nothing beyond Lean's three standard axioms is used — in particular no compiled evaluation (`Lean.ofReduceBool`), so every computation above, including the generated proof, was done by the kernel itself.

```
'StepZK.StepCert.eval_iff' depends on axioms: [propext, Quot.sound]
'StepZK.certOf_meets' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.Trace.eval_iff' depends on axioms: [propext, Quot.sound]
'StepZK.Trace.acceptedIdx_length_eq_iff' depends on axioms: [propext, Quot.sound]
'StepZK.sound_chain_replays' depends on axioms: [propext, Quot.sound]
'StepZK.chain_WF' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.chain_length_le' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.chain_defects_le' depends on axioms: [propext, Quot.sound]
'StepZK.selfHost_of_certified' depends on axioms: [propext, Quot.sound]
'StepZK.selfHost_certified' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.selfHost_stable_certified' depends on axioms: [propext, Quot.sound]
'StepZK.audit_iter_sound' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.stepZK_two_level' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.demo_trace_sound' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.demo_self_hosted' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.demo_forgery_rejected' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.demo_digests' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.traceGates_satisfiable_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.stepR1CS_satisfiable_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.Certificate.stepVec_eq_digests' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.Certificate.stepCert_verifies' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.Certificate.stepCert_data' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.Certificate.stepCert_extraction' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.Certificate.stepCert_hides_run' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.Generated.generated_proof' depends on axioms: [propext, Classical.choice, Quot.sound]
'StepZK.stepZK_end_to_end' depends on axioms: [propext, Classical.choice, Quot.sound]
```

12 Reproducing this paper

Every number above is printed by `lake exe stepzk`, which runs the same definitions the theorems cite, and every displayed statement is extracted verbatim from the sources:

```
lake build stepzk
./.lake/build/bin/stepzk > papers/stepzk_artifacts.json
./.lake/build/bin/stepzk --proof > RequestProject/StepZK/Generated.lean
lake build RequestProject.StepZK.Audit   # the kernel re-checks it all
python3 scripts/stepzk_paper.py          # writes this PDF
```

Or, in one step: `make stepzk-paper`.